

OjaMind

Autonomous Offline AI Business Operating System for African SMEs

Version 1.0

Table of Contents

1. Executive Summary

- Project overview
 - Vision
 - Mission
 - Why OjaMind exists
 - Expected impact
 - One-page elevator pitch
-

2. The Problem

The African SME Problem

- Informal economy
 - Internet dependence
 - Software cost
 - Inflation
 - Manual bookkeeping
 - Inventory losses
 - Fraud
 - Lack of insights
 - Low digital adoption
-

3. Why Existing Solutions Fail

Examples:

- Expensive SaaS
 - Monthly subscriptions
 - Cloud dependency
 - Complex interfaces
 - Poor support for African businesses
 - Poor offline capability
 - No conversational interaction
-

4. Proposed Solution

What is OjaMind?

How does it solve the problem?

Why offline?

Why AI?

Why voice?

5. Product Philosophy

Design principles such as:

- Offline first
- AI where ambiguity exists
- Deterministic business logic
- Privacy by default
- Zero subscriptions
- Runs on African hardware
- Low RAM
- Low CPU
- Explainable AI

6. Target Users

Primary

- Supermarkets
- Kiosks
- Provision stores

Secondary

- Pharmacies
- Building material shops
- Agro dealers

Future

- Warehouses
 - SMEs
 - Schools
-

7. Core Features

Inventory

Sales

Credit Ledger

Voice Recording

Natural Language Queries

Dashboard

Reports

Audit

Business Intelligence

Profit Tracking

Restocking

Price Management

8. AI Capabilities

Exactly what AI does.

Exactly what AI DOES NOT do.

This is important.

9. System Architecture

Large architecture diagram.

Every component explained.

Frontend

↓

API

↓

Whisper

↓

LLM

↓

Intent Layer

↓

Validation Layer



Business Rule Engine



SQLite



Dashboard

10. AI Pipeline

Voice



Speech-to-text



Intent extraction



JSON schema



Validation



Business Rules



SQL



Response

11. LLM Design

Chosen model

Alternatives

Prompt engineering

Temperature

Context

Memory

Why Qwen?

Why not Phi?

Why not Gemma?

Why not Llama?

12. Speech Recognition

Whisper

Alternatives

Optimization

Supported languages

Noise handling

13. Structured Intent Specification

Define every intent.

Example:

Record Sale

Record Purchase

Restock

Inventory Query

Profit Query

Credit Query

Daily Report

Delete Item

Modify Item

Customer Lookup

Everything.

14. JSON Schema

Every response format.

Examples.

Validation rules.

Confidence.

15. Validation Engine

What it checks.

Missing fields

Invalid quantities

Invalid dates

Unknown products

Duplicate customers

Confidence threshold

16. Inventory Resolver

Product matching

Aliases

Synonyms

Ambiguity handling

Follow-up questions

17. Business Rules Engine

Probably one of the largest sections.

Rules.

Inventory

Credit

Pricing

Discounts

Profit

Negative stock

Duplicate names

Expiry

Returns

Refunds

Manager approvals

Overrides

Warnings

18. Warning Engine

Non-blocking errors.

Examples.

19. Blocker Engine

Blocking errors.

Examples.

20. Audit Engine

Every action logged.

Every AI correction logged.

Every override logged.

Every warning logged.

21. SQLite Design

ER Diagram

Tables

Relationships

Indexes

Transactions

Normalization

22. Database Schema

Inventory

Sales

Purchases

Customers

Credit

Employees

Settings

Audit Logs

23. API Design

Every endpoint.

POST

GET

DELETE

PATCH

Audio upload

Transaction

Reports

24. Frontend Design

Pages

Dashboard

Sales

Inventory

Reports

Analytics

Settings

Voice

25. User Journey

Shopkeeper

Cashier

Manager

Admin

26. Memory Optimization Strategy

Critical.

Expected RAM.

Whisper

LLM

SQLite

FastAPI

Browser

OS

Target

<6GB

27. Performance Targets

Latency

Inference

Speech

Transactions

28. Security

Offline

Encryption

Local users

Passwords

Database protection

29. Failure Recovery

Power failure

Corrupt database

Wrong speech

Wrong inventory

Rollback

30. Risks

Technical

Business

Team

Hardware

Competition

31. Future Features

OCR

Barcode

Receipt

WhatsApp

Synchronization

Mobile app

African languages

32. Development Roadmap

Week by week.

Sprint 1

Sprint 2

Sprint 3

Sprint 4

33. Task Allocation

You

AI

Backend

Architecture

Documentation

Testing

Prince

Frontend

UI

UX

Charts

Third Member

Testing

Documentation

Deployment

Benchmarking

34. Testing Strategy

Unit

Integration

Voice

Stress

Memory

Performance

35. Benchmark Strategy

How we maximize:

Accuracy

Speed

Efficiency

Based directly on the challenge scoring.

36. Demo Script

Exactly what judges see.

Minute 1

Minute 2

Questions

Fallback

37. Commercialization

Business model.

Future.

Customers.

Pricing.

Deployment.

38. Open Source Strategy

Repository structure.

Documentation.

Licensing.

39. Final Submission Checklist

Everything required for Gate 1.

One suggestion I strongly recommend

Since this is likely to become the largest technical project you've worked on, don't write it as a Markdown README.

Instead, let's produce it as a professional Software Architecture & Product Design Specification with diagrams, UML, flowcharts, sequence diagrams, ER diagrams, API contracts, JSON schemas, state diagrams, memory budgets, and implementation notes.

My recommendation is to target 70–100 pages. That may sound excessive, but this document would become:

- Your development guide.
- Your onboarding manual.
- The basis for your **REPORT.md** submission.
- A source for your pitch deck.
- A foundation for future commercialization if OjaMind evolves beyond the hackathon.

Given the complexity of this project, I believe that level of documentation will pay dividends throughout development.